

7-4 · MinIO·Qdrant 연동 · 이론 1H

# 오브젝트 스토리지와 Vector DB의 역할 분담

참고와 카드함 — 원본은 어디에, 색인은 어디에

수도권ICT 이노베이션스퀘어  
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

# 1시간 뒤에는

---

- ① 원본 보관(MinIO)과 검색 색인(Qdrant)의 역할 분담을 설명한다
- ② 색인을 「언제든 다시 만들 수 있는 파생물」로 설계하는 이유를 안다
- ③ payload 의 source 로 원본 문서를 역참조하는 구조를 그린다
- ④ 포인트 id 를 uuid5(경로 + 조각번호)로 정해 재실행을 안전하게 한다

다음 1H 실습에서 MinIO 를 띄우고 파이썬으로 버킷·객체를 직접 다룹니다

0. 4주차에서 이어집니다

# 같은 저장소를 두 방향으로 — 정형 위에 비정형



「레이크하우스에 모든 것을 모은다」는 말의 실제 모습 — 한 저장소, 두 쓰임새

MinIO 는 AWS S3 와 같은 API 를 쓰는 S3 호환 스토리지 — 로컬에서 배운 코드가 클라우드에서 그대로 통한다

## 1. 왜 나누는가

# 참고와 카드함 — 카드에서 찾고, 선반 번호로 원본을 꺼낸다



카드함에는 「이 내용이 참고 어느 선반에 있다」는 요약 카드(벡터 + 출처)만 꽂아 둔다

1. 왜 나누는가

# MinIO(창고)와 Qdrant(카드함) — 여섯 가지 차이

|       | MinIO (창고) | Qdrant (카드함)     |
|-------|------------|------------------|
| 담는 것  | 원본 문서 전체   | 조각 + 벡터 + 출처     |
| 크기    | 크다 (GB~TB) | 작다               |
| 정보 여부 | 정보         | 파생물 — 다시 만들 수 있다 |
| 수정·삭제 | 여기서 한다     | 재적재로 따라간다        |
| 권한 관리 | 여기서 한다     | 보조               |
| 찾는 방식 | 경로를 알아야 한다 | 뜻으로 찾는다          |

원본의 정보는 한 곳에만 두어야 수정·삭제·권한 관리가 한 번에 된다

## 1. 왜 나누는가

# 원문 전체를 벡터DB 에 넣지 않는 이유

1

### 정보는 한 곳에만

수정·삭제·권한 관리를 한 곳에서 한 번에 끝낸다

2

### 색인은 파생물

날려도 원본에서 다시 만든다 — 모델 교체·파라미터 변경이 두렵지 않다

3

### source 역참조

payload 에 minio://버킷/키 를 남기면 원본 전체를 근거로 다시 꺼낸다

설계 원칙

색인은 버리고  
다시 만든다

이 원칙이 지켜지면 chunk\_size 변경·임베딩 모델 교체가 스크립트 재실행 한 번이 된다

## 2. 역등한 적재

# 같은 스크립트를 두 번 돌려도 같아야 — 포인트 id 가 답

## 순번 · uuid4() — 재실행이 위험

돌릴 때마다 결과가 달라진다

- 0, 1, 2 ... 순번: 문서가 바뀌면 엉뚱한 조각을 덮어씀
- uuid4() 무작위: 중복이 계속 쌓임
- 4주차 MERGE 에서 본 것과 같은 문제

VS

## uuid5(경로 + 조각번호) — 권장

같은 입력이면 항상 같은 UUID (해시 기반)

- `uuid5(NAMESPACE_URL, "minio://raw-docs/notes/notes.txt#0")`
- 몇 번을 돌려도 같은 값
- `upsert` 는 같은 id 면 덮어쓴다 → 재실행이 안전

## 2. 멍등한 적재

# 멍등이 완벽하지는 않다 — 문서가 줄면 옛 조각이 남는다

- 1 문서를 수정해 조각 수가 줄면**  
앞부분 조각은 같은 id 로 덮어쓰이지만, 예전 뒷부분 조각은 그대로 남는다
- 2 검색하면 지워진 내용이 나온다**  
실무에서 가장 골치 아픈 종류의 버그 — 눈에 잘 띄지 않는다
- 3 실습 마지막에 직접 재현한다**  
문서를 줄여 다시 올려 보고, 해결까지 해 본다

1.5H 실습에서

구멍 재현과  
해결까지

# 이 1시간에서 꼭 가져갈 것

---

- 1 원본은 스토리지에, 색인은 벡터DB 에 둔다
- 2 색인은 파생물이므로 언제든지 다시 만들 수 있어야 한다
- 3 payload 의 source 로 원본을 역참조한다
- 4 포인트 id 를 '경로 + 조각번호'에서 만들면 재실행이 안전하다
- 5 문서가 줄어들 때는 옛 조각 정리가 따로 필요하다

#### 7-4 · 이론 1H 마무리

## 다음은 MinIO 버킷 구성과 문서 적재 실습 (1H)

MinIO 를 띄우고 파이썬으로 버킷·객체를 다루며, 일부러 HTTPS 로 붙어 SSL 오류도 직접 봅니다

✓ `docker run` 으로 MinIO 기동 · 콘솔 9001 로그인

✓ `raw-docs` 버킷 생성과 `notes` 문서 두 개 업로드

✓ `secure=True` 로 붙어 SSL 오류 보기

✓ `get_object` · 없는 키 · `prefix` 로 훑기

준비할 것

7-3 Qdrant 컨테이너 · 9000·9001 포트 비우기